

DSA STUDY BLUEPRINT SERIES

46. Quick Sort Algorithm

In-place Pivot Partitioning Sorting (Quick Sort)

Section 06 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Pivot Choice:** Pick pivot; arrange elements such that lesser are on left, greater on right.
- Perform sorting recursively on partitioned subsets.

Memory & Execution Blueprint

[Elements < Pivot] ← **[Pivot]** → [Elements > Pivot]

C++ Implementation Reference

Standard code layout and syntax

```
int partition(int arr[], int l, int r) {
    int pivot = arr[r], i = l - 1;
    for (int j = l; j < r; j++) {
        if (arr[j] < pivot) swap(arr[++i], arr[j]);
    }
    swap(arr[i + 1], arr[r]);
    return i + 1;
}

void quickSort(int arr[], int l, int r) {
    if (l < r) {
        int pi = partition(arr, l, r);
        quickSort(arr, l, pi - 1);
        quickSort(arr, pi + 1, r);
    }
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Average Case	$O(N \log N)$ time, $O(\log N)$ space
Worst Case	$O(N^2)$ (Already sorted inputs)

Key Practices & Gotchas

- **Important:** Randomly shuffling input arrays helps avoid worst-case $O(N^2)$ scenarios.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's dry run the partition splitting sequence for Merge Sort on list `[4, 2, 1, 3]` :

Depth Level	Divided segments	Sorting result	Merged result
1	[4, 2] & [1, 3]	Recursive sorting	[2, 4] & [1, 3] merged to [1, 2, 3, 4]

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Pivot heuristics:** Quick Sort speeds depend on pivot balance; choosing middle or random elements prevents $O(N^2)$ degradation.
- **Inversions:** Count array inversions during merge sorts to identify out-of-order bounds sizes.
- **Related LeetCode Problems:** #912 Sort an Array, #315 Count of Smaller Numbers After Self.